



UNIVERSITAT POLITÈCNICA
DE CATALUNYA
BARCELONATECH

KNOWLEDGE GRAPHS

SDM Second Assignment

Stefano Galiano, Pablo Rodríguez

13-05-2026

Contents

1	Section A — Modeling	1
1.1	Overview	1
1.2	Metamodel Layer	2
1.3	Model Layer	2
1.4	Facts Layer	3
1.5	Design Justifications	3
1.6	RDFS Knowledge Graph vs. Neo4j Property Graph	4
2	Section B — Implementation	5
3	Section C — Querying	6
3.1	Assumptions	6
3.2	Query 1 — List All Drugs	6
3.3	Query 2 — List All Diseases	7
3.4	Query 3 — Drug–Disease Pairs Where Drug Affects Disease	7
3.5	Summary of Inferred Triple Patterns	9

1 Section A — Modeling

1.1 Overview

This section describes the RDFS knowledge graph designed to model relationships between drugs and diseases in a biomedical domain. The graph is organized in three canonical layers:

1. **Metamodel layer** — the RDF/RDFS vocabulary constructs used to define classes and properties.
2. **Model layer** — the domain taxonomy (class hierarchy and property hierarchy).
3. **Facts layer** — the instances (individual drugs and diseases) and the explicit relationships between them.

The two root entities are `bio:Drug` and `bio:Disease`. Drugs are specialized into *AntiInflammatory*, *Antibiotic*, *Analgesic*, *Antiviral*, and *Steroid*. Diseases are specialized into *InflammatoryDisease*, *InfectiousDisease*, *ChronicDisease*, and *NeurologicalDisease*. A single top-level property `bio:affects` is refined into three subproperties: `treats`, `relieves`, and `worsens`.

The namespace used throughout is:

`http://www.example.org/biomedical#` (prefix `bio:`)

Here is a picture of the metamodel and model layers, as well as one example of an instance:

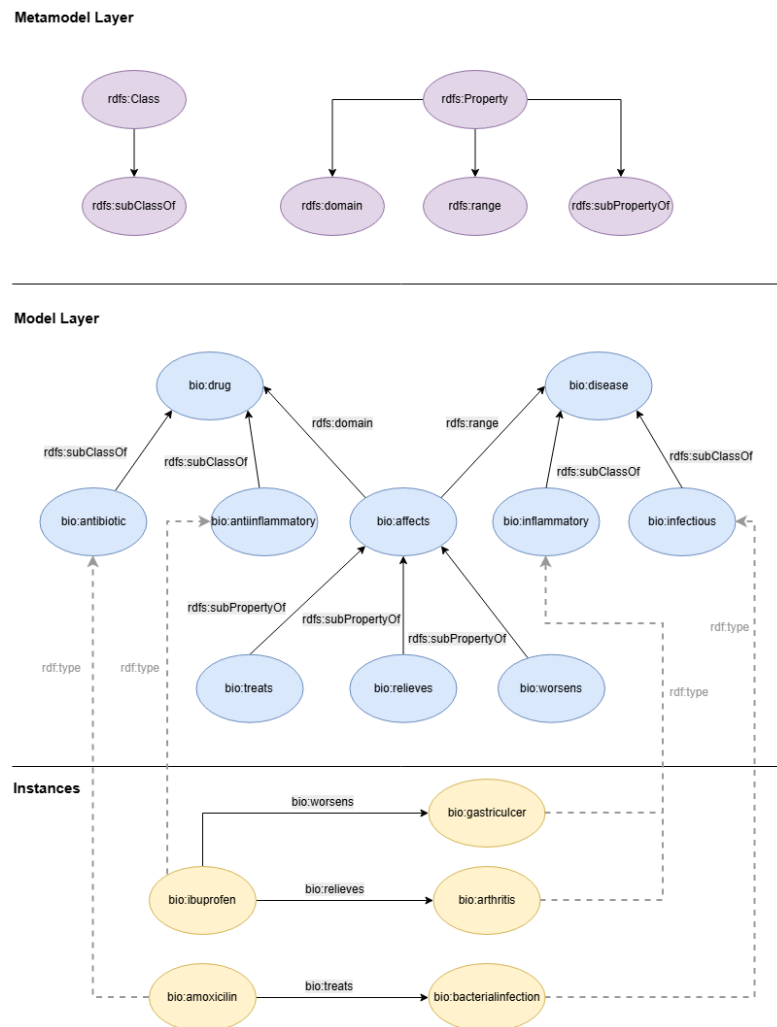


Figure 1: Metamodel, model and instance layers of the graph

1.2 Metamodel Layer

The metamodel layer uses only standard RDF/RDFS vocabulary. It expresses *how classes and properties are constructed*, independently of the domain content. The key constructs employed are listed in Table 1.

Table 1: RDF/RDFS constructs used in the metamodel layer.

Construct	Role in this graph
<code>rdfs:Class</code>	Declares <code>bio:Drug</code> , <code>bio:Disease</code> , and all subclasses as RDF classes.
<code>rdfs:subClassOf</code>	Establishes the class hierarchy (e.g., <code>AntiInflammatory rdfs:subClassOf Drug</code>).
<code>rdf:Property</code>	Declares <code>bio:affects</code> , <code>bio:treats</code> , <code>bio:relieves</code> , <code>bio:worsens</code> as properties.
<code>rdfs:subPropertyOf</code>	Establishes the property hierarchy (e.g., <code>treats rdfs:subPropertyOf affects</code>).
<code>rdfs:domain</code>	Constrains the subject of <code>affects</code> to be a <code>Drug</code> .
<code>rdfs:range</code>	Constrains the object of <code>affects</code> to be a <code>Disease</code> .
<code>rdf:type</code>	Assigns each instance to exactly one leaf class (e.g., <code>Ibuprofen rdf:type AntiInflammatory</code>).

Note that `rdfs:domain` and `rdfs:range` are declared only on `bio:affects`; the subproperties inherit them through the property hierarchy. Likewise, `rdf:type` arrows from model-layer elements to metamodel-layer elements (e.g. `bio:Drug rdf:type rdfs:Class`) are not asserted explicitly — they are derived from RDFS built-in axioms. Both decisions are justified in Section 1.5.

The metamodel layer is illustrated in the top section of Figure 1.

1.3 Model Layer

The model layer defines the *domain-specific* taxonomy: which classes exist and how they are related, and which properties constitute the vocabulary.

Class Hierarchy

```
bio:Drug
  bio:AntiInflammatory
  bio:Antibiotic

bio:Disease
  bio:InflammatoryDisease
  bio:InfectiousDisease
```

Each subclass is declared with `rdfs:subClassOf` pointing to its immediate parent. The implementation extends this base schema with additional subclasses to reach 100 instances; see Section 2 for details.

Property Hierarchy

```
bio:affects      (domain: bio:Drug | range: bio:Disease)
  bio:treats      -- subPropertyOf bio:affects
  bio:relieves    -- subPropertyOf bio:affects
```

```
bio:worsens      -- subPropertyOf bio:affects
```

1.4 Facts Layer

Instances

The graph contains exactly **100 instances**: 50 drugs spread across the five drug subclasses, and 50 diseases spread across the four disease subclasses. Table 2 summarizes the counts.

Table 2: Instance distribution across leaf classes.

Root class	Subclass	Count
Drug	AntiInflammatory	10
	Antibiotic	10
	Analgesic	10
	Antiviral	10
	Steroid	10
Disease	InflammatoryDisease	13
	InfectiousDisease	13
	ChronicDisease	12
	NeurologicalDisease	12
Total		100

Explicit Relationship Triples

Each instance is typed to exactly one leaf class using `rdf:type`. Relationship triples use only the three specific subproperties (`treats`, `relieves`, `worsens`) — the `affects` property is *never asserted explicitly* between instances. GraphDB’s RDFS reasoner will derive all `affects` triples automatically via the `subPropertyOf` rule. A sample of the explicit facts is shown below:

Listing 1: Sample Turtle triples from the facts layer.

bio:Ibuprofen	rdf:type	bio:AntiInflammatory .
bio:Ibuprofen	bio:relieves	bio:Arthritis .
bio:Ibuprofen	bio:worsens	bio:ChronicKidneyDisease
bio:Amoxicillin	rdf:type	bio:Antibiotic .
bio:Amoxicillin	bio:treats	bio:Pneumonia .

In total the graph contains 76 explicit relationship triples (29 `treats`, 33 `relieves`, 14 `worsens`) and 205 triples overall including the metamodel and model declarations.

1.5 Design Justifications

Why a property hierarchy for `affects`? A single abstract property `affects` unifies the three specific relationships. This lets users write a single SPARQL query for “all drug–disease interactions” without knowing whether the relationship is therapeutic, symptomatic, or contraindicated. Without this hierarchy, a query would need a `UNION` of three triple patterns; with it, querying `bio:affects` suffices.

Why minimise explicit triples? To rely on RDFS inference rather than asserting every derivable triple. By never asserting `bio:Ibuprofen rdf:type bio:Drug` explicitly; GraphDB derives it from `Ibuprofen rdf:type AntiInflammatory` and `AntiInflammatory rdfs:subClassOf Drug`. This approach reduces graph size, eliminates the risk of inconsistent redundancies.

Why `rdfs:domain` and `rdfs:range` only on `affects`? Declaring constraints at the most general level and relying on inheritance prevents repeating them on each subproperty. RDFS does not mandate that subproperties inherit domain/range, but GraphDB’s RDFS (Optimized) ruleset does apply the standard RDFS-entailment rules (`rdfs4`, `rdfs7`, `rdfs9`) so this works as expected.

1.6 RDFS Knowledge Graph vs. Neo4j Property Graph

Key gap: the capability that RDFS provides and Neo4j does not is *ontological inference* — the ability to derive new facts from structural constraints (subclass, subproperty, domain, range) without rewriting queries or duplicating data. A Neo4j property graph can store the same raw facts, but it cannot answer “find all drugs that affect any disease” without a programmer explicitly enumerating every relationship type in the query. Explicit `rdfs:subClassOf` chain; querying a parent class returns all subclass instances automatically. Node labels are flat; no inheritance. A query for `Drug` returns only nodes with that exact label — not `Antibiotic` nodes unless explicitly listed. Facts are stated once at the most specific level; all generalizations are inferred. Every fact that should be queryable must be explicitly present. A new drug class requires updating all queries that should match it.

2 Section B — Implementation

The base schema defined in Section A contains two drug subclasses (`AntiInflammatory`, `Antibiotic`) and two disease subclasses (`InflammatoryDisease`, `InfectiousDisease`). To reach the required 100 instances, the implementation extends the schema with three additional drug subclasses — `Analgesic`, `Antiviral`, `Steroid` — and two additional disease subclasses — `ChronicDisease`, `NeurologicalDisease`. All five new subclasses are declared with `rdfs:subClassOf` pointing to their respective root class, following the same pattern as the original schema.

The knowledge graph has five logical blocks:

1. **Metamodel declarations** — `rdfs:Class`, `rdfs:subClassOf`, `rdf:Property`, `rdfs:subPropertyOf`, `rdfs:domain`, `rdfs:range`.
2. **Instance typing** — one `rdf:type` triple per instance pointing to the most specific (leaf) class.
3. **Relationship triples** — `treats`, `relieves`, `worsens` facts derived from clinical knowledge.
4. **Serialisation** — `g.serialize(format="turtle")` writes `biomedical_kg.ttl`.
5. **GraphDB upload** — waits for the server, creates the repository using a Turtle configuration file posted as a multipart form to `/rest/repositories`, then uploads the triples via POST `/repositories/biomedical/statements`.

We created the script by the **RDFS (Optimized)** ruleset (`rdfsplus-optimized`), which is GraphDB’s full RDFS entailment regime that materialises inferred triples at load time.

We added 100 instances that follow triple counts as it is seen in Table 3

Table 3: Triple counts in the generated graph.

Category	Count
Drug instances (<code>rdf:type</code> leaf class)	50
Disease instances (<code>rdf:type</code> leaf class)	50
<code>treats</code> relationships	29
<code>relieves</code> relationships	33
<code>worsens</code> relationships	14
Metamodel and model declarations	29
Total triples in .ttl file	205

3 Section C — Querying

3.1 Assumptions

- GraphDB is running with the **RDFS (Optimized)** ruleset (`rdfsplus-optimized`), which materialises inferred triples at load time so that SPARQL queries over the default graph see both explicit and inferred triples.
- The equivalent Neo4j graph stores the same 100 nodes and 76 relationships, but uses flat labels (`:Drug`, `:AntiInflammatory`, ...) and typed relationships (`:TREATS`, `:RELIEVES`, `:WORSENS`) with no schema hierarchy. Every node that should match a “drug” query carries both the specific label (e.g. `:Antibiotic`) and the general label (`:Drug`) — these must be added explicitly by the data loader; they are not derived.
- SPARQL queries use the namespace prefix `bio:` `<http://www.example.org/biomedical#>`.
- `SELECT DISTINCT` is used throughout to suppress any duplicate bindings that could arise from multiple inferred paths to the same triple.

3.2 Query 1 — List All Drugs

Listing 2: SPARQL: list all drugs.

```
PREFIX bio: <http://www.example.org/biomedical#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>

SELECT DISTINCT ?drug
WHERE {
    ?drug rdf:type bio:Drug .
}
ORDER BY ?drug
```

Listing 3: Cypher equivalent: list all drugs.

```
MATCH (d:Drug)
RETURN d.name AS drug
ORDER BY drug
```

Comparison with Neo4j

The surface syntax is very similar, but the *semantics* differ fundamentally. In the SPARQL version, `bio:Drug` is a class in an RDFS hierarchy; querying it returns all instances of any subclass without listing them. In the Cypher version, `:Drug` is a plain label — it only matches nodes that carry exactly that label. The Cypher query returns all 50 drug nodes because each was manually labeled `:Drug`. If a new drug subclass is added later, the query will only work if the `:Drug` label is also applied — but Neo4j has no schema mechanism to enforce this. In SPARQL, adding a new subclass (`bio:Antifungal rdfs:subClassOf bio:Drug`) is sufficient; the query needs no modification.

Inferences applied by GraphDB

The query pattern `?drug rdf:type bio:Drug` triggers RDFS rule **rdfs9**, which states: if `?u rdf:type ?C` and `?C rdfs:subClassOf ?D`, then infer `?u rdf:type ?D`. None of the 50 drug instances are explicitly typed as `bio:Drug` in the uploaded Turtle file; they are typed only as their leaf class. GraphDB materialises the following inferred triples (one per instance, five subclass chains):


```

bio:Ibuprofen      rdf:type  bio:Drug .  # via AntiInflammatory
    subClassOf Drug
bio:Amoxicillin    rdf:type  bio:Drug .  # via Antibiotic
    subClassOf Drug
bio:Paracetamol    rdf:type  bio:Drug .  # via Analgesic
    subClassOf Drug
# ... 45 more (one per remaining instance)

```

3.3 Query 2 — List All Diseases

Listing 4: SPARQL: list all diseases.

```

PREFIX bio: <http://www.example.org/biomedical#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>

SELECT DISTINCT ?disease
WHERE {
    ?disease rdf:type bio:Disease .
}
ORDER BY ?disease

```

Listing 5: Cypher equivalent: list all diseases.

```

MATCH (d:Disease)
RETURN d.name AS disease
ORDER BY disease

```

Comparison with Neo4j

Identical structural argument as Query 1. The Cypher query works only because every disease node was given the `:Disease` label in addition to its specific label (`:InflammatoryDisease`, etc.) at load time. The SPARQL query works because RDFS subclass inference makes it unnecessary to assert the general type explicitly. The two queries produce the same result set, but one relies on inference while the other relies on redundant labelling.

Inferences applied by GraphDB

The same rdfs9 rule applies across four subclass chains:

```

bio:Arthritis      rdf:type  bio:Disease .  # via
    InflammatoryDisease subClassOf Disease
bio:Pneumonia      rdf:type  bio:Disease .  # via InfectiousDisease
    subClassOf Disease
bio:Diabetes        rdf:type  bio:Disease .  # via ChronicDisease
    subClassOf Disease
# ... 46 more

```

3.4 Query 3 — Drug–Disease Pairs Where Drug Affects Disease

Listing 6: SPARQL: all drug–disease interaction pairs.

```

PREFIX bio: <http://www.example.org/biomedical#>

SELECT DISTINCT ?drug ?disease
WHERE {
    ?drug bio:affects ?disease .
}

```

```
}
ORDER BY ?drug ?disease
```

Listing 7: Cypher equivalent: all drug–disease interactions.

```
MATCH (d)-[r:TREATS|RELIEVES|WORSENS]->(dis)
RETURN d.name AS drug, dis.name AS disease
ORDER BY drug, disease
```

Comparison with Neo4j

This is the query where the two models diverge most sharply. In SPARQL, a single triple pattern on `bio:affects` retrieves all 76 pairs — `bio:affects` is never explicitly used in the data, but RDFS infers it from the three subproperties. In Cypher, there is no concept of a property superset; the query must enumerate all three relationship types using the `|` union syntax. If a fourth relationship type (e.g. `:PREVENTS`) were added, the Cypher query would silently omit it unless the developer remembered to update the pattern. The SPARQL query would include it automatically, provided only that the new property is declared `rdfs:subPropertyOf bio:affects`.

A further difference is expressiveness: Neo4j has no way to write a query equivalent to “find all drug–disease pairs connected by *any* relationship that is a subtype of `affects`” without explicitly naming the subtypes. RDFS makes this possible through a single abstract property.

Inferences applied by GraphDB

Two rules cooperate here.

Rule `rdfs7` (subproperty propagation): Applied to every explicit relationship triple, this produces 76 inferred `bio:affects` triples — one for each asserted `treats/relieves/worsens` fact. Examples:

```
# Explicit:
bio:Ibuprofen      bio:relieves  bio:Arthritis .
bio:Amoxicillin    bio:treats     bio:Pneumonia .
bio:Ibuprofen      bio:worsens    bio:ChronicKidneyDisease .

# Inferred via rdfs7:
bio:Ibuprofen      bio:affects    bio:Arthritis .
bio:Amoxicillin    bio:affects    bio:Pneumonia .
bio:Ibuprofen      bio:affects    bio:ChronicKidneyDisease .
```

Rules `rdfs2` and `rdfs3` (domain and range typing): since `bio:affects` has `rdfs:domain bio:Drug` and `rdfs:range bio:Disease`, GraphDB additionally materialises:

```
# For every subject of an affects triple (via rdfs2):
bio:Ibuprofen      rdf:type  bio:Drug .
bio:Amoxicillin    rdf:type  bio:Drug .

# For every object of an affects triple (via rdfs3):
bio:Arthritis      rdf:type  bio:Disease .
bio:Pneumonia      rdf:type  bio:Disease .
```

Note that these domain/range inferences *overlap* with the `rdfs9` subclass inferences from Queries 1 and 2. Both paths produce the same `rdf:type bio:Drug` and `rdf:type bio:Disease` triples, which GraphDB deduplicates. This redundancy is harmless and actually desirable: it means the typing is robust even if one inference path were to fail.

3.5 Summary of Inferred Triple Patterns

Table 4: All inferred triple patterns, the RDFS rules that produce them, and a concrete example for each.

Inferred pattern	Rule	Concrete example
<code>?x rdf:type bio:Drug</code>	<code>rdfs9</code>	<code>Ibuprofen rdf:type Drug</code> derived from <code>Ibuprofen</code> <code>rdf:type AntiInflammatory</code> and <code>AntiInflammatory subClassOf Drug</code> .
<code>?x rdf:type bio:Disease</code>	<code>rdfs9</code>	<code>Arthritis rdf:type Disease</code> de- rived from <code>Arthritis rdf:type</code> <code>InflammatoryDisease</code> and <code>InflammatoryDisease subClassOf</code> <code>Disease</code> .
<code>?d bio:affects ?dis</code>	<code>rdfs7</code>	<code>Ibuprofen affects Arthritis</code> derived from <code>Ibuprofen relieves Arthritis</code> and <code>relieves subPropertyOf affects</code> .
<code>?d rdf:type bio:Drug</code> (from domain)	<code>rdfs2</code>	<code>Amoxicillin rdf:type Drug</code> derived from <code>Amoxicillin treats Pneumonia</code> and <code>affects rdfs:domain Drug</code> (cas- cades via <code>rdfs7</code>).
<code>?dis rdf:type bio:Disease</code> (from range)	<code>rdfs3</code>	<code>Pneumonia rdf:type Disease</code> derived from <code>Amoxicillin treats Pneumonia</code> and <code>affects rdfs:range Disease</code> .

All five patterns behave as expected. The RDFS (Optimized) ruleset in GraphDB implements the complete RDFS entailment regime. Every inferred triple is materialised into the repository before query evaluation begins, so query performance is not affected by the number of inference steps required.